PRESENTER COPY | 5-MINUTE PITCH | JUDGE PREPARATION

RELAY AI

Pitch Script and Judge Q&A

Matches the 14-slide deck and the Detailed Project Documentation. Dated 04 November 2026.

Honesty rule. Say only what is true on the day. Text in [brackets] must be replaced with the real value shown on screen. Never state that tests passed unless the Testing Agent actually ran them in that run. Verify event rules, Switch setup and deadlines against official sources before presenting.

1. Pitch at a glance

Time	Segment	Speaker	Slides	Length
0:00-0:20	Opening	Member 4	1	20 s
0:20-1:20	Problem	Member 4	2, 3 (brief)	60 s
1:20-2:20	Solution	Member 3	4, 5, 8	60 s
2:20-4:10	Live demo	Member 3 drives, Member 2 narrates	9, then live product	110 s
4:10-4:40	Technical differentiation	Member 1	6, 7, 10	30 s
4:40-5:00	Closing	Member 4	13, 14	20 s

Slides 11 (business value) and 12 (differentiation) are held for Q&A.; Fourteen slides cannot be explained in five minutes; hide 3, 11 and 12 in the slideshow if time is tight, and keep them ready for questions.

2. Speaker allocation and what each member can defend

Member	Speaks on	Can defend (questions to take)
Member 1: Backend and Architecture Lead	Technical differentiation (slides 6, 7, 10)	Orchestrator, state machine, API, database, scalability, cost, security architecture
Member 2: AI and Agent Engineer	Demo narration (agent behaviour)	Prompts, schemas, hallucination control, agent reliability, why multi-agent
Member 3: Frontend and Integration Engineer	Solution, Switch, demo driver	Switch integration, dashboard, GitHub integration, user experience
Member 4: Testing, DevOps and Presentation Engineer	Opening, problem, closing	Docker sandbox, test evidence, demo reliability, backup plan, business case

3. Slide-to-speaker mapping

Slide	Title / message	Speaker	Use
1	Cover: RELAY AI, tagline	Member 4	Opening
2	Problem: one bug, five tools	Member 4	Problem
3	Why existing approaches fall short	Member 4	One sentence only, or hidden
4	Solution: reviewable evidence	Member 3	Solution
5	How it works: eight stages	Member 3	Solution
6	Multi-agent architecture	Member 1	Technical differentiation
7	Technical architecture (proposed stack)	Member 1	Technical differentiation
8	The Switch advantage	Member 3	Solution (15 s)
9	Live product experience	Member 3 / Member 2	Demo lead-in, then live product
10	Security and human approval	Member 1	Technical differentiation
11	Business value and users	Member 4	Q&A;
12	Competitive differentiation	Member 1	Q&A;
13	Team and roadmap	Member 4	Closing
14	Closing	Member 4	Closing

4. Timed script components (read in order = the complete 5-minute pitch)

Approximate speaking rate: 150 words per minute. Each block below is spoken exactly as written; stage directions are in blue.

A. Opening: 20 seconds. Member 4. Slide 1

A teammate types one line into chat: "checkout total looks wrong." Then the scramble starts. Who investigates? Where is the code? Who runs the tests? Today that coordination lives across chat, repositories, terminals and people's memory. We built RELAY AI so that one message becomes an investigated, tested change, and a human makes the final call.

B. Problem: 60 seconds. Member 4. Slides 2 and 3

For a small team, a bug report is not one task, it is a chain. Someone clarifies it in chat. Someone searches the repository. A developer writes a fix locally, runs tests in a terminal, and a reviewer checks it somewhere else. Every handoff loses context, and nobody has one view of who is doing what. When an AI suggests a fix, it often arrives without test evidence or a clear approval step, so the team re-verifies it anyway. *[Slide 3]* Chat, trackers, AI assistants and CI each solve one piece. Nobody owns the whole path from report to reviewed fix. That is the gap: not another chatbot, but a workflow where every step leaves evidence.

C. Solution: 60 seconds. Member 3. Slides 4, 5 and 8

RELAY AI connects a Switch-connected team room, a GitHub repository and five specialist agents under one orchestrator. *[Slide 5]* A teammate reports a bug. The Triage Agent turns it into a structured task. The Investigator Agent searches the repository and cites the files it relies on. The Developer Agent proposes a minimal patch in an isolated workspace. The Testing Agent runs the tests in a Docker sandbox and records the real output. The Review Agent checks the diff and flags risks. Then everything stops. A human approves or rejects. Only after approval is a pull request created and a report posted back to the room. *[Slide 8]* Switch is the shared context: the team and the agents work in one room, and results return there.

D. Live demo: 90 to 120 seconds (planned 110 s). Member 3 drives, Member 2 narrates

Goal: show one seeded bug travelling from report to human approval with real evidence. Flash slide 9 for 5 seconds, then switch to the product.

Time	Action on screen	Narration	Expected result
0:00-0:12	Show demo store and the failing regression test	"This demo store has one seeded bug: a discount is applied twice. This test fails today."	Failing test output visible [actual failing test name]
0:12-0:27	Post the bug in the Switch-connected room	"A teammate reports it in the team room."	Task REL-001 appears with severity and acceptance criteria
0:27-0:45	Open Investigator result	"The Investigator cites the exact function it suspects, and lists what it is unsure about."	File and function reference [src/checkout.py, function name]; uncertainty listed
0:45-1:00	Open Developer diff	"The Developer proposes the smallest change, in an isolated workspace, not on main."	Diff panel with changed lines
1:00-1:15	Open Testing Agent report	"Real tests, in a container. Here is the command, the exit code and the output."	Command, exit code, pass/fail counts [read the real numbers]
1:15-1:30	Open Review Agent findings	"The Review Agent checks the diff against the evidence and flags remaining risk."	Findings and recommendation [read real findings]
1:30-1:50	Click Approve (or Reject); show the room	"Nothing happens until a human decides. On approval, a pull request is created and the report returns to the room."	Status APPROVED then PR_CREATED; PR link and report posted [only if P1 PR step is working]

If the pull request step is not implemented, stop at "approved, ready for pull request" and say so plainly. Do not show a mock PR as real.

E. Technical differentiation: 30 seconds. Member 1. Slides 6, 7 and 10

Under the hood, a FastAPI orchestrator owns an explicit state machine. Each agent gets only the context it needs and must return schema-validated JSON; bad output is retried or stops the workflow. Tests run in a disposable Docker container with no network, never on a host machine. The GitHub token is least-privilege, and the approval gate is enforced in the backend, not just the interface. We report a test result only when a command actually ran.

F. Closing: 20 seconds. Member 4. Slides 13 and 14

RELAY AI is one small workflow that genuinely works: one bug report in, one reviewed, tested change out. Agents do the work; humans hold the decision. We are four builders with clear ownership, and we would be glad to show you the repository and the evidence. Thank you.

Transitions. A to B: "Let us make that scramble concrete." B to C: "So we built the missing owner for that path." C to D: "Let us watch it happen." D to E: "What makes this trustworthy?" E to F: "To close."

5. Backup demo plan

Rule: anything replayed is labelled "recorded run" aloud. Never present cached output as live.

If this fails	Fallback	What to say
LLM API (quota, outage)	Replay mode: cached agent outputs from a previous real run behind the same LLM adapter	"The model API is unavailable, so this is a replay of a real earlier run through the same pipeline."
Switch or chat event	Submit the same bug through a dashboard form that calls the same event endpoint	"Switch is not responding right now; the same event comes in through our API."
Docker sandbox	Show the stored test report and logs from the last real run	"Here is the recorded test run from earlier."
GitHub API	Show the generated branch and diff locally; stop at approval	"PR creation depends on the GitHub API; the approval gate is the key step."

If this fails	Fallback	What to say
Network or laptop failure	Play the pre-recorded demo video and show screenshots from the same task	"We will play the recorded run of the same workflow."

- Before presenting: run the full workflow once from a clean state; pre-pull the sandbox image; confirm API keys and quotas; verify the Switch room; close unrelated windows.
- Keep the recorded video, screenshots and the cached-run fixtures on the presenting laptop and one backup device.
- Decide in advance who calls the switch to backup (Member 3) and who talks while it loads (Member 2).

6. Judge questions and honest answers

Before the event, update answers marked "[fill]" with what is actually implemented. Short, truthful answers beat confident ones.

Technical feasibility

1. Can a four-person student team really build this in a hackathon?

We scoped it so we can: one repository, one language, one seeded bug, five agents in a fixed workflow, SQLite and one LLM provider. P0 items come first; GitHub pull requests and multi-task views are P1 and get cut if needed. We will state exactly what is implemented. [fill]

2. Is the demo bug staged?

Yes. It is deliberately seeded and reproducible, with a regression test that fails before the fix. We do not claim RELAY AI reliably fixes arbitrary production bugs. Our evidence is repeatable runs on seeded scenarios: [fill number of runs and results].

3. What if the generated patch is wrong?

It can be. That is why the workflow has real test execution, a Review Agent and a human approval gate. A passing test suite is evidence, not proof, and we show failures and uncertainty rather than hiding them.

Agent reliability

4. How do you stop agents hallucinating?

We cannot eliminate it. We reduce it: agents must cite files and snippets, return schema-validated JSON, and the orchestrator rejects invalid output. Claims without evidence are labelled hypotheses, and the human reviews the final evidence.

5. Why multiple agents instead of one?

To bound context and tools per stage, make each step auditable, and let the orchestrator validate every handoff. The trade-off is added latency and model calls. The value is accountability and evidence, not a claim that more agents is automatically smarter.

6. What happens when the model returns malformed output?

The orchestrator validates against the schema, retries a bounded number of times, then marks the task BLOCKED with the error. It never advances silently.

7. Does a passing test prove the fix is correct?

No. It shows the tested behaviour passes. The Review Agent and human look for what tests miss, and coverage of our demo repository limits what a pass proves.

Switch integration

8. How central is Switch?

Switch is the shared room where the report arrives, agents share context and results return. We use only the capabilities in official Switch documentation: [fill exact Switch features used]. Anything beyond that is not claimed.

9. What if the Switch integration fails during a demo?

The chat connector sits behind a small interface (receive event, post message). Our fallback submits the same event through our API, and we say so openly.

10. Could this work without Switch?

Architecturally yes, through another connector. For the Flip the Switch entry, Switch is the point, so we show it live. [fill]

Security

11. What stops an agent from running something dangerous?

Model output never runs on a host. Tests run in a disposable container with no network, resource limits, non-root user and an allow-list of commands. Docker is not a perfect security boundary; for production we would add stronger isolation.

12. What about prompt injection in bug reports or repository files?

We treat that text as data, not instructions, and agents cannot request arbitrary commands because tools are constrained. We do not claim immunity; we test with malicious inputs and keep the human approval gate.

13. How are secrets handled?

Server-side environment variables, an untracked .env file, a least-privilege token limited to the demo repository, and no secrets in prompts, logs or chat.

Business viability

14. Who would pay for this?

We have not validated willingness to pay. Our hypothesis is small software teams that lose time on coordination. Next steps would be interviews and a small pilot, and we make no revenue or market claims.

15. Why would this survive if bigger platforms build similar things?

They might. Our bet is a design that is evidence-first, approval-gated and lives in the team's shared room. We have not tested that against any specific product and would not claim to.

Scalability

16. Does it work on large repositories or monorepos?

Not in the MVP. Larger codebases would need indexing and scoped retrieval, plus more language and build-system support. We chose a narrow scope on purpose.

17. How would it handle many tasks at once?

The MVP processes tasks sequentially with SQLite. Scaling would need a job queue, a production database and one sandbox container per task. That is future work.

Cost

18. What does one run cost?

We do not know until we measure. We log model usage per run so we can report it; cost depends mainly on repository context size and the chosen provider. [fill measured value if available]

Competitors

19. How is this different from other AI coding agents?

We have not benchmarked against other products, so we do not claim to be better. Our design differences are: a structured task record, evidence at every stage, tests actually executed and recorded, and a mandatory human approval gate in a shared team room.

Implementation honesty

20. What did you actually build versus propose?

[Fill truthfully before pitching.] Implemented: [list]. Partially working: [list]. Proposed only: [list]. Our documentation labels everything proposed unless it is implemented.

7. Final pre-pitch checklist

- Official rules, deadline, track and Switch setup verified; slides and README use identical project terminology.
- Three consecutive successful rehearsals; timed at 5:00 or less.
- Every [bracket] in this script replaced with real values; Q&A; items marked [fill] updated.
- Demo repository contains no secrets; API keys and quotas confirmed.
- Backup video, screenshots and replay fixtures on the presenting laptop and a second device.

- Each member can explain their component in 30 seconds and knows which questions they take.
- Limitations stated honestly in the pitch or Q&A.;